



Documentação Técnica - Integração TSL Denver



Índice

1. [Visão Geral](#visão-geral)
 2. [Especificação de Interfaces](#especificação-de-interfaces)
 3. [Arquitetura do Projeto](#arquitetura-do-projeto)
 4. [Componentes Implementados](#componentes-implementados)
 5. [Queries e Estrutura de Dados](#queries-e-estrutura-de-dados)
 6. [Otimizações de Performance](#otimizações-de-performance)
 7. [Utilização de Utilitários Sankhya](#utilização-de-utilitários-sankhya)
 8. [Guia de Uso](#guia-de-uso)
 9. [Formatação e Regras](#formatação-e-regras)
 10. [Observações Importantes](#observações-importantes)
-



Visão Geral

Informações do Projeto

- **Cliente**: Denver
- **Projeto**: Integração Total Service Logística (TSL)
- **Documento Base**: Interfaces Padrões WMS - TSL V1.2
- **Versão**: 1.0.0
- **Status**:  **CONCLUÍDO**

Objetivo

Implementar integração completa entre o sistema Sankhya e o Total Service Logística (TSL), gerando arquivos de exportação no formato posicional conforme manual de interfaces padrões WMS versão 1.2, contemplando:

1. **REC_IN** - Recebimento de Mercadorias Integrado
2. **PED_IN** - Expedição de Mercadorias

Especificações Técnicas dos Arquivos

- **Formato**: Arquivo texto posicional (campos fixos)
- **Encoding**: ANSI (Windows-1252)
- **Quebra de linha**: PC/Windows (CRLF - `\r\n`)
- **Separador de campos**: Espaço em branco ao final de cada campo
- **Codificação**: Posicional (cada campo possui posição e tamanho fixo)

Nomenclatura dos Arquivos

Formato: `{INTERFACE}_{CNPJ}_{TIMESTAMP}.txt`

Onde:

- **INTERFACE**: Tipo de interface (`REC\_IN` ou `PED\_IN`)
- **CNPJ**: CNPJ da empresa (14 dígitos, sem formatação)
- **TIMESTAMP**: Data e hora da criação no formato `yyyyMMddHHmmss`

Exemplo:

- `REC\_IN\_12345678000190\_20250101120000.txt`
- `PED\_IN\_12345678000190\_20250101120000.txt`



Especificação de Interfaces

REC_IN - Recebimento de Mercadorias Integrado

Tamanho da linha: 619 caracteres

Campo	Tamanho	Tipo	Obrigatório	Descrição
-------	---------	------	-------------	-----------

CNPJ	15	N	S	CNPJ da empresa (14 dígitos + espaço)
Nota Fiscal	13	N	S	Número da nota fiscal
Item da NF	7	N	S	Sequência do item na nota fiscal
Número do Palete	26	AN	N	Número identificador do palete
Código do Produto	51	AN	S	Código de referência do produto
Identificador da caixa	31	AN	N	Identificador da caixa
Peso da Caixa	18	N	S	Peso com 2 casas decimais (vírgula)
Data de Produção	11	DT	N	Data formato DD/MM/YYYY
Data de Vencimento	11	DT	N	Data formato DD/MM/YYYY
Lote	26	AN	N	Código do lote
Informação Complementar	401	AN	N	Informações adicionais
Valor Unitário	9	N	S	Valor com 5 inteiros e 2 decimais (vírgula)

Total: 619 caracteres

PED_IN - Expedição de Mercadorias

Tamanho da linha: 232 caracteres

Campo	Tamanho	Tipo	Obrigatório	Descrição
CNPJ	15	N	S	CNPJ da empresa (14 dígitos + espaço)
Ordem de Frete	21	AN	N	Número da ordem de frete
Número do Pedido	13	N	S	Número do pedido/nota fiscal
Item do pedido	7	N	S	Sequência do item no pedido
Código do produto	51	AN	S	Código de referência do produto
Número do Palete	26	AN	N	Número identificador do palete
Quantidade	17	N	S	Quantidade com 2 casas decimais (vírgula)
Peso	19	N	S	Peso total com 2 casas decimais (vírgula)
Data de Fabricação (DE)	11	DT	N	Data formato DD/MM/YYYY
Data de Fabricação (ATÉ)	11	DT	N	Data formato DD/MM/YYYY

Lote	26	AN	N	Código do lote
CNPJ Cliente	15	N	S	CNPJ do cliente (14 dígitos + espaço)

Total: 232 caracteres

Arquitetura do Projeto

Estrutura de Diretórios

```

Denver/
├── Docs/
│   ├── Documento - Plano de requisitos (Denver) (1).pdf
│   └── Interfaces Padro_es WMS - TSL V1.2.pdf
├── docs/
│   └── DOCUMENTACAO_TECNICA.md ✓ Este arquivo
├── lib/
│   ├── jape.jar
│   ├── mge-modelcore.jar
│   ├── sanutil.jar
│   └── SankhyaW-extensions.jar
├── pom.xml
├── README.md
└── REFATORACAO_PERFORMANCE.md

```

```

├─ REVISAO_QUERIES.md
├─ src/
│   └─ br/com/denver/tsl/
│       ├── action/botaoAcao/
│       │   └─ GerarArquivoTSL.java ✓ Botão de ação principal
│       ├── model/dto/
│       │   ├── RecebimentoDTO.java ✓ DTO para recebimentos
│       │   └─ ExpedicaoDTO.java ✓ DTO para expedições
│       ├── repository/
│       │   ├── AbstractTSLRepository.java ✓ Repositório base
│       │   ├── RecebimentoRepository.java ✓ Query de recebimentos
│       │   └─ ExpedicaoRepository.java ✓ Query de expedições
│       ├── service/
│       │   └─ TSLService.java ✓ Serviço principal
│       └─ util/
│           ├── FileGenerator.java ✓ Geração de arquivos
│           ├── TSLConstants.java ✓ Constantes do projeto
│           └─ TSLFormatter.java ✓ Formataadores de campos

```

Padrão Arquitetural

O projeto segue o padrão **MVC (Model-View-Controller)** adaptado para integração:

- **Model**: DTOs (`RecebimentoDTO`, `ExpedicaoDTO`)
- **Repository**: Camada de acesso a dados (queries SQL)
- **Service**: Lógica de negócio e orquestração
- **Action**: Camada de apresentação (botão de ação Sankhya)
- **Util**: Utilitários e constantes

Stack Tecnológico

- **Java JDK 8**
 - **Maven 3.x**
 - **Sankhya Core** (via JARs locais)
 - `SankhyaW-extensions.jar`
 - `jape.jar`
 - `mge-modelcore.jar`
 - `sanutil.jar`
 - **Oracle Database** (via JDBC do Sankhya)
-

Componentes Implementados

1. Action Layer

`GerarArquivoTSL.java`

- **Responsabilidade**: Interface com o usuário através de botão de ação
- **Funcionalidades**:
 - Captura de parâmetros de período (DATA_INICIO e DATA_FIM)
 - Processamento automático de ambas interfaces (REC_IN e PED_IN)
 - Tratamento de erros e mensagens de retorno
- **Parâmetros Configuráveis**:
 - `DATA\_INICIO`: Data inicial do período (Timestamp) - Opcional

- `DATA\_FIM`: Data final do período (Timestamp) - Opcional
- **Valores Fixos**:
- Tipo de interface: "TODOS" (sempre gera REC_IN e PED_IN)
- Caminho de exportação: Diretório temporário do sistema

2. Service Layer

`TSLService.java`

- **Responsabilidade**: Orquestração da geração de arquivos
- **Métodos**:
- `gerarArquivoRecebimento()`: Gera arquivo REC_IN
- `gerarArquivoExpedicao()`: Gera arquivo PED_IN
- `gerarLinhaRecebimento()`: Formata linha REC_IN
- `gerarLinhaExpedicao()`: Formata linha PED_IN

3. Repository Layer

`AbstractTSLRepository.java`

- **Responsabilidade**: Base para repositórios com lógica comum
- **Funcionalidades**:
- Execução de queries SQL
- Gestão de recursos (JDBC, ResultSet)
- Filtros de data
- Conversão de tipos

``RecebimentoRepository.java``

- **Query**: Busca recebimentos (TIPMOV = 'C', STATUSNOTA = 'L')
- **Tabelas**: TGFCAB, TGFITE, TGFPPO, TSIEMP, TGFEST
- **Filtros**: Data de movimento, produtos ativos, estoque ativo

``ExpedicaoRepository.java``

- **Query**: Busca expedições (TIPMOV = 'V', STATUSNOTA = 'L')
- **Tabelas**: TGFCAB, TGFITE, TGFPPO, TGFPAR, TSIEMP, TGFEST
- **Filtros**: Data de movimento, produtos ativos, estoque ativo

4. Model Layer

``RecebimentoDTO.java``

- DTO para dados de recebimento
- Campos alinhados com interface REC_IN

``ExpedicaoDTO.java``

- DTO para dados de expedição
- Campos alinhados com interface PED_IN

5. Util Layer

``TSLConstants.java``

- Constantes do projeto

- Tamanhos de linha, separadores, encoding

``TSLFormatter.java``

- Formataadores para todos os campos
- Cache de espaços em branco
- ThreadLocal para formataadores de data
- Otimizações de performance

``FileGenerator.java``

- Geração de arquivos TXT
- Buffer otimizado (32KB)
- Encoding Windows-1252
- Criação automática de diretórios

Queries e Estrutura de Dados

Recebimento (REC_IN)

Tabelas Utilizadas

- ``TGFCAB``: Cabeçalho de notas fiscais
- ``TGFITE``: Itens de notas fiscais
- ``TGFPRO``: Produtos

- `TSIEMP`: Empresas
- `TGFEST`: Estoque (para datas de fabricação/validade)

Query Principal

```
SELECT
    REPLACE(REPLACE(REPLACE(REPLACE(EMP.CGC, '.', ''), '/', ''), '-', ''), ' ', '') AS CGC,
    CAB.NUMNOTA AS NOTAFISCAL,
    ITE.SEQUENCIA AS ITEMNOTA,
    NVL(PRO.REFERENCIA, LPAD(TO_CHAR(PRO.CODPROD), 13, '0')) AS CODIGOPRODUTO,
    NVL(PRO.PESOLIQ, 0) AS PESOCAIXA,
    EST.DTFABRICACAO AS DATAPRODUCAO,
    EST.DTVAL AS DATAVENCIMENTO,
    ITE.CONTROLE AS LOTE,
    PRO.COMPLDESC AS INFOCOMPLEMENTAR,
    NVL(ITE.VLRUNIT, 0) AS VALORUNITARIO,
    PRO.AD_NUMEROPALETE AS NUMEROPALETE,
    PRO.AD_IDENTIFICADORCAIXA AS IDENTIFICADORCAIXA
FROM TGFCAB CAB
INNER JOIN TGFITE ITE ON ITE.NUNOTA = CAB.NUNOTA
INNER JOIN TGFPRO PRO ON PRO.CODPROD = ITE.CODPROD AND PRO.ATIVO = 'S'
    AND PRO.AD_INTEGTOTALLOGISTICA = 'S'
INNER JOIN TSIEMP EMP ON EMP.CODEMP = CAB.CODEMP
LEFT JOIN TGFEST EST ON EST.CODPROD = ITE.CODPROD
    AND EST.CODEMP = CAB.CODEMP
    AND ((EST.CONTROLE IS NULL AND ITE.CONTROLE IS NULL) OR EST.CONTROLE = ITE.CONTROLE)
    AND EST.ATIVO = 'S'
WHERE CAB.TIPMOV = 'C' AND CAB.STATUSNOTA = 'L'
```

Filtros Adicionais

- Período: `CAB.DTNEG >= :DATAINI AND CAB.DTNEG <= :DATAFIM` (aplicado dinamicamente)

- Produtos ativos: `PRO.ATIVO = 'S'`
- Produtos para integração: `PRO.AD\_INTEGTOTALLOGISTICA = 'S'` ★ ****NOVO****
- Estoque ativo: `EST.ATIVO = 'S'`

Campos Adicionais Utilizados

- ****AD_INTEGTOTALLOGISTICA****: Filtro para exportar apenas produtos marcados ('S')
- ****AD_NUMEROPALETE****: Número do palete (campo adicional na TGFPRO)
- ****AD_IDENTIFICADORCAIXA****: Identificador da caixa (campo adicional na TGFPRO)

Tratamento de Valores Nulos

- ****CODIGOPRODUTO****: `NVL(REFERENCIA, LPAD(TO\_CHAR(CODPROD), 13, '0'))`
- Fallback para CODPROD com zeros à esquerda
- ****PESOCAIXA****: `NVL(PESOLIQ, 0)` - Fallback para zero
- ****VALORUNITARIO****: `NVL(VLRUNIT, 0)` - Fallback para zero

Expedição (PED_IN)

Tabelas Utilizadas

- `TGFCAB`: Cabeçalho de notas fiscais
- `TGFITE`: Itens de notas fiscais
- `TGFPRO`: Produtos
- `TGFPAR`: Parceiros (clientes)
- `TSIEMP`: Empresas
- `TGFEST`: Estoque (para data de fabricação)

Query Principal

```
SELECT
  REPLACE(REPLACE(REPLACE(REPLACE(EMP.CGC, '.', ''), '/', ''), '- ', ''), ' ', '') AS CODIGOEMP,
  CAB.ORDEMCARGA AS ORDEMFRETE,
  CAB.NUMNOTA AS NUMEROPEDIDO,
  ITE.SEQUENCIA AS ITEMPEDIDO,
  NVL(PRO.REFERENCIA, LPAD(TO_CHAR(PRO.CODPROD), 13, '0')) AS CODIGOPRODUTO,
  ABS(ITE.QTDNEG) AS QUANTIDADE,
  NVL(PRO.PESOLIQ, 0) * ABS(ITE.QTDNEG) AS PESO,
  EST.DTFABRICACAO AS DATAFABRICACAODE,
  EST.DTFABRICACAO AS DATAFABRICACAOATE,
  ITE.CONTROLE AS LOTE,
  REPLACE(REPLACE(REPLACE(REPLACE(NVL(PAR.CGC_CPF, ''), '.', ''), '/', ''), '- ', ''), ' ', '') AS CODIGOPAR,
  PRO.AD_NUMEROPALETE AS NUMEROPALETE
FROM TGFCAB CAB
INNER JOIN TGFITE ITE ON ITE.NUNOTA = CAB.NUNOTA
INNER JOIN TGFPRO PRO ON PRO.CODPROD = ITE.CODPROD AND PRO.ATIVO = 'S'
  AND PRO.AD_INTEGTOTALLOGISTICA = 'S'
INNER JOIN TGFPAR PAR ON PAR.CODPARC = CAB.CODPARC
INNER JOIN TSIEMP EMP ON EMP.CODEMP = CAB.CODEMP
LEFT JOIN TGFEST EST ON EST.CODPROD = ITE.CODPROD
  AND EST.CODEMP = CAB.CODEMP
  AND ((EST.CONTROLE IS NULL AND ITE.CONTROLE IS NULL) OR EST.CONTROLE = ITE.CONTROLE)
  AND EST.CODLOCAL = ITE.CODLOCALORIG
  AND EST.ATIVO = 'S'
WHERE CAB.TIPMOV = 'V' AND CAB.STATUSNOTA = 'L'
```

Filtros Adicionais

- Período: `CAB.DTNEG >= :DATAINI AND CAB.DTNEG <= :DATAFIM` (aplicado dinamicamente, Data de Negociação)
- Produtos ativos: `PRO.ATIVO = 'S'`
- Produtos para integração: `PRO.AD\_INTEGTOTALLOGISTICA = 'S'` ★ **NOVO**

- Estoque ativo: `EST.ATIVO = 'S'`

Campos Adicionais Utilizados

- ****AD_INTEGTOTALLOGISTICA****: Filtro para exportar apenas produtos marcados ('S')
- ****AD_NUMEROPALETE****: Número do palete (campo adicional na TGFPRO)

Tratamento de Valores Nulos

- ****CODIGOPRODUTO****: `NVL(REFERENCIA, LPAD(TO\_CHAR(CODPROD), 13, '0'))`
- Fallback para CODPROD com zeros à esquerda
- ****PESO****: `NVL(PESOLIQ, 0) \* ABS(QTDNEG)` - Fallback para zero na multiplicação
- ****CNPJCLIENTE****: `NVL(CGC\_CPF, '')` - Fallback para string vazia (formatação preenche com espaços)

Otimizações de Query

1. ****Uso de REFERENCIA****: Prioriza `PRO.REFERENCIA` (código de barras/EAN) com fallback para `CODPROD` com zeros à esquerda (13 dígitos)
 2. ****Filtro de Produtos Ativos****: Apenas produtos com `PRO.ATIVO = 'S'`
 3. ****Filtro de Integração****: ★ ****NOVO**** - Apenas produtos com `PRO.AD\_INTEGTOTALLOGISTICA = 'S'`
 4. ****Join Otimizado com TGFEST****: Tratamento correto de NULL para comparação de lotes
 5. ****ABS() para Quantidade****: Garante valores positivos mesmo em devoluções
 6. ****Fallbacks de Valores Nulos****: Todos os campos obrigatórios têm tratamento adequado
-

Otimizações de Performance

Cache e Memória

1. **Cache de Espaços**

- `ConcurrentHashMap` para strings de espaços
- `ConcurrentHashMap` para arrays de caracteres
- Evita alocações repetidas

1. **ThreadLocal para Formatadores**

- `SimpleDateFormat` por thread
- Evita sincronização e problemas de concorrência
- 2 formatadores: data (dd/MM/yyyy) e timestamp (yyyyMMddHHmmss)

1. **Pré-alocação de Collections**

- `ArrayList` com capacidade inicial de 1024
- Reduz realocações durante crescimento

Manipulação de Strings

1. **Arrays de Char Diretos**

- Uso de `char[]` para preenchimento
- `Arrays.fill()` para eficiência
- `getChars()` para cópia direta

1. ****StringBuilder Pré-dimensionado****

- Capacidade inicial igual ao tamanho final da linha
- Evita realocações durante construção

1. ****Pattern Compilado****

- Regex compilada uma única vez
- Reutilizada para limpeza de CNPJ/CPF

I/O Otimizado

1. ****Buffer Aumentado****

- `BufferedWriter` com 32KB (32768 bytes)
- Reduz chamadas ao sistema operacional

1. ****Escrita Direta****

- Arrays de char para separadores de linha
- Evita criação de strings intermediárias

1. ****Try-with-Resources****

- Fechamento automático de recursos
- Garante liberação mesmo em caso de erro

Processamento de Dados

1. ****Formatação Eficiente****

- Uso de `Locale.US` para números
- Conversão direta ponto/vírgula
- Validações mínimas

1. ****Validações Inline****

- Verificações apenas quando necessário
- Early return para casos nulos/vazios

Banco de Dados

1. ****Queries Otimizadas****

- Uso adequado de `INNER JOIN` e `LEFT JOIN`
- Filtros aplicados no banco (WHERE)
- Índices nativos do Sankhya

1. ****Native SQL****

- Uso de `NativeSql` do Sankhya
- Performance máxima
- Parâmetros nomeados

1. ****Gestão de Recursos****

- Fechamento adequado em blocos `finally`
- Tratamento de exceções silencioso para cleanup

Utilização de Utilitários Sankhya

Utilitários Utilizados

1. **StringUtils** (`com.sankhya.util.StringUtils`)

- `getNullAsEmpty()`: Trata valores nulos
- `isEmpty()`: Verifica strings vazias

1. **JdbcWrapper** (`br.com.sankhya.jape.dao.JdbcWrapper`)

- Gerenciamento de conexões JDBC
- Sessões de banco de dados

1. **NativeSql** (`br.com.sankhya.jape.sql.NativeSql`)

- Execução de queries SQL nativas
- Parâmetros nomeados
- Gestão de recursos

1. **EntityFacadeFactory** (`br.com.sankhya.modelcore.util.EntityFacadeFactory`)

- Acesso ao DWF (Dynamic Web Framework)
- Facilita acesso a dados

1. **AcaoRotinaJava**

(`br.com.sankhya.extensions.actionbutton.AcaoRotinaJava`)

- Interface para botões de ação

- Integração com interface do Sankhya

Guia de Uso

Instalação

1. **Compilar o projeto**:

```
cd Denver  
mvn clean package
```

1. **Copiar JAR gerado**:

- Localização: `target/integracao-tsl-1.0.0.jar`
- Destino: Diretório de extensões do Sankhya

1. **Configurar Botão de Ação no Sankhya**:

- **Tela**: Configuração → Rotinas → Botões de Ação
- **Classe**: `br.com.denver.tsl.action.botaoAcao.GerarArquivoTSL`
- **Método**: `doAction`
- **Parâmetros opcionais**:
 - `DATA\_INICIO`: Data inicial do período (Timestamp)
 - `DATA\_FIM`: Data final do período (Timestamp)

Configuração de Parâmetros

Apenas os períodos são configuráveis via parâmetros do botão de ação:

Parâmetro	Tipo	Obrigatório	Padrão	Descrição
`DATA_INICIO`	Timestamp	Não	`null`	Data inicial do período (se não informado, processa todos os registros)
`DATA_FIM`	Timestamp	Não	`null`	Data final do período (se não informado, processa todos os registros)

Valores Fixos:

- `TIPO\_INTERFACE`: Fixo em "TODOS" (gera REC_IN e PED_IN automaticamente)
- `CAMINHO\_EXPORTACAO`: Fixo no diretório temporário do sistema (`java.io.tmpdir`)

Uso

1. Acessar a tela onde o botão foi configurado
2. Clicar no botão "Gerar Arquivo TSL"
3. Informar os períodos (opcional - se não informado, processa todos os registros)
4. Aguardar processamento
5. Verificar mensagem de retorno com localização dos arquivos

Exemplo de Uso

Com filtro de período:

```
DATA_INICIO: 2025-01-01 00:00:00
```

```
DATA_FIM: 2025-01-31 23:59:59
```

Sem filtro (processa todos os registros):

```
DATA_INICIO: (vazio/null)
```

```
DATA_FIM: (vazio/null)
```

Resultado esperado:

- `REC\_IN\_12345678000190\_20250131123045.txt`
- `PED\_IN\_12345678000190\_20250131123045.txt`



Formatação e Regras

Regras de Formatação

Datas

- **Formato**: `DD/MM/YYYY`
- **Exemplo**: `01/01/2025`
- **Campo vazio**: Preenchido com espaços

Números Decimais

- **Separador decimal**: Vírgula (`,`)
- **Exemplo**: `125,50`
- **Valor Unitário**: 5 inteiros + 2 decimais (ex: `00125,50`)
- **Peso/Quantidade**: Formato livre com 2 decimais

CNPJ/CPF

- **Formato**: Apenas números (14 dígitos)
- **Preenchimento**: Direita com espaços até 15 caracteres
- **Exemplo**: `12345678000190 ` (com espaço no final)

Campos Alfanuméricos

- **Preenchimento**: Direita com espaços
- **Truncamento**: Se exceder tamanho, corta à direita

Campos Numéricos (sem decimal)

- **Preenchimento**: Esquerda com zeros
- **Exemplo**: `0001234`

Validações

1. ****Tamanho de Linha****

- REC_IN: Exatamente 619 caracteres
- PED_IN: Exatamente 232 caracteres
- Linhas com tamanho incorreto são descartadas

1. ****Campos Obrigatórios****

- CNPJ, Nota Fiscal, Item, Código Produto
- Validação via SQL (campos não nulos)

1. ****Formato de Datas****

- Validação automática no formato
- Campos nulos geram espaços em branco

Observações Importantes

Dados do Banco

1. ****Produtos Ativos****: Apenas produtos com `PRO.ATIVO = 'S'` são processados
2. ****Produtos para Integração****:  ****NOVO**** - Apenas produtos com `PRO.AD\_INTEGTOTALLOGISTICA = 'S'` são exportados
3. ****Notas Liberadas****: Apenas notas com `STATUSNOTA = 'L'` são consideradas

4. ****Estoque Ativo****: Datas de fabricação/validade apenas de estoque ativo
5. ****Referência do Produto****: Prioriza `PRO.REFERENCIA`, fallback para `LPAD(TO\_CHAR(PRO.CODPROD), 13, '0')` (13 dígitos com zeros à esquerda)

Campos Adicionais Necessários

O projeto utiliza os seguintes campos adicionais na tabela `TGFPRO` :

Campo	Tipo	Valores	Descrição
`AD_INTEGTOTALLOGISTICA`	CHAR(1)	'S', 'N' ou NULL	Flag que indica se o produto deve ser exportado. Apenas produtos com 'S' são processados.
`AD_NUMEROPALETE`	VARCHAR2	Texto (26 caracteres)	Número identificador do palete (opcional, usado em REC_IN e PED_IN)
`AD_IDENTIFICADORCAIXA`	VARCHAR2	Texto (31 caracteres)	Identificador da caixa (opcional, usado apenas em REC_IN)

 **Importante:** Os campos adicionais devem ser criados na tabela `TGFPRO` antes de usar o sistema. Produtos sem `AD_INTEGTOTALLOGISTICA = 'S'` não serão exportados.

Arquivos Gerados

1. **Encoding**: Windows-1252 (ANSI), não UTF-8
2. **Quebra de Linha**: CRLF (`\r\n`)
3. **Formato**: Posicional (campos fixos)
4. **Validação**: Linhas com tamanho incorreto são ignoradas

Performance

1. **Grandes Volumes**: Sistema otimizado para processar grandes volumes
2. **Memória**: Pré-alocação reduz uso de memória
3. **I/O**: Buffer de 32KB otimiza escrita em disco
4. **Cache**: Cache de strings reduz alocações

Limitações

1. **Período**: Filtro por data de movimento (`DTNEG`)
2. **Empresa**: Processa apenas uma empresa por vez
3. **Produtos para Integração**: Apenas produtos com
`AD\_INTEGTOTALLOGISTICA = 'S'` são exportados
4. **Lotes Nulos**: Tratamento adequado para produtos sem lote
5. **Campos Adicionais**: Requer campos adicionais criados na tabela `TGFPRO`

Tratamento de Campos Obrigatórios

Todos os campos obrigatórios das interfaces têm tratamento de valores nulos:

- **Código do Produto**: Se `REFERENCIA` for NULL, usa `CODPROD` formatado com zeros à esquerda (13 dígitos)
- **Peso**: Se `PESOLIQ` for NULL, usa `0`
- **Valor Unitário**: Se `VLRUNIT` for NULL, usa `0`
- **CNPJ Cliente** (PED_IN): Se `CGC\_CPF` for NULL, usa string vazia (formatador preenche com espaços)

Troubleshooting

Nenhum registro encontrado

- Verificar filtros de data
- Verificar status das notas (`STATUSNOTA = 'L'`)
- Verificar produtos ativos (`PRO.ATIVO = 'S'`)
- ★ **Verificar se produtos têm `AD\_INTEGTOTALLOGISTICA = 'S'`** (campo adicional obrigatório)
- Verificar se campos adicionais foram criados na tabela `TGFPRO`

Arquivo não gerado

- Verificar permissões no diretório de saída
- Verificar espaço em disco
- Verificar logs de erro

Dados incorretos

- Verificar formato dos campos
- Verificar tamanho das linhas
- Verificar encoding do arquivo

Referências

- **Manual TSL**: Interfaces Padrões WMS - TSL V1.2
 - **Documento de Requisitos**: Documento - Plano de requisitos (Denver)
 - **Sankhya**: Documentação oficial do Sankhya
 - **JDK 8**: Oracle Java Documentation
-

Versão: 1.0.0

Última Atualização: 2025-12-01

Status:  CONCLUÍDO